TEORÍA DE COMPILADORES

StockLang

Lenguaje para describir inventarios

Hito 2 · Segundo avance

José David Antonio Morán · U202223356
Sebastian Cordova Montenegro · U202321420

Universidad Peruana de Ciencias Aplicadas
Ciencias de la Computación · Sección 4889
Profesor: José Luis Soncco Alvarez · 2026-2

INVENTARIO EN ESPAÑOL

```
producto lapiz stock 10;
entrada lapiz 5;
mostrar lapiz;
```

Stock final: 15

Problema y objetivo del proyecto

El inventario necesita reglas claras

Un producto inexistente, una cantidad inválida o un retiro excesivo pueden producir un inventario incoherente.

Comprobar antes de ejecutar

01 **Reconocer programas**

Construir el compilador con ANTLR4.

02 **Ampliar la semántica**

Incorporar sino y siete clases de error.

03 **Contrastar resultados**

Comparar el stock calculado con la VM.

Construcciones del lenguaje

Construcción	Propósito	Ejemplo independiente
Declaración	Crear un producto	producto lapiz stock 100;
Movimiento	Ingresar o retirar unidades	entrada lapiz 10;
Ajuste	Asignar un nuevo stock	ajustar lapiz = 80;
Consulta	Mostrar el stock	mostrar lapiz;
Condicional	Elegir un bloque	si lapiz < 10 { entrada lapiz 20; }
Expresión	Calcular una cantidad	(4 + 6) * 2

Las declaraciones aparecen primero. Los bloques contienen al menos una instrucción.

Léxico y modelo de datos

Qué reconoce el lexer

Familia	Ejemplos
Reservadas	producto, stock, entrada, sino
Comparadores	< <= > >= == !=
Operadores	= + - *
Identificadores	lapiz, item_2
Omitidos	Espacios y comentarios //

L001 identifica un carácter sin regla léxica.

0 a 2 147 483 647

Rango permitido para el stock

Reglas relevantes

Los ID distinguen mayúsculas.
Se aplica la coincidencia más larga.
Las reservadas ganan en empate.
Las posiciones empiezan en 1.

Gramática y precedencia

Estructura principal

```
Programa → Declaración
  ListaDeclaraciones
  ListaInstrucciones EOF

Condicional → si Condición
  Bloque Alternativa
Alternativa → sino Bloque | ε
```

Precedencia de operaciones

1. **Paréntesis**
   (4 + 6) * 2
2. **Multiplicación**
   2 * 3 + 1
3. **Suma y resta**
   cuaderno - 2

Asociación por la izquierda. El informe documenta 24 derivaciones en el anexo.

Arquitectura del compilador

driver.py recibe el archivo. src/compiler.py coordina las fases.

01

Lexer

Tokens y posiciones

02

Parser

Árbol sintáctico

03

Semántica

Símbolos y stock válido

04

Generador IR

Operaciones en JSON

05

VM

Inventario y consultas

Un diagnóstico bloquea la fase posterior

AnalysisResult conserva tokens, árbol, símbolos, diagnósticos, stock, IR y ejecución.

Los siete controles semánticos

Código	Regla	Ejemplo inválido
E001	Producto duplicado	Declarar dos veces el producto a
E002	Producto no declarado	mostrar b; sin declarar b
E003	Stock inicial fuera de rango	producto a stock 2147483648;
E004	Movimiento no positivo	entrada a 0;
E005	Ajuste negativo	ajustar a = 1 - 2;
E006	Stock insuficiente	Con stock 10, ejecutar salida a 11;
E007	Desbordamiento entero	Con stock máximo, ejecutar entrada a 1;

Cualquier error semántico impide generar IR y ejecutar el programa.

El condicional si / sino

```
producto a stock 10;
si a < 0 {
  ajustar a = 100;
} sino {
  salida a 9;
}
mostrar a;
```

Estado inicial

a = 10

La condición a < 0 es falsa

La rama sino retira 9 unidades.

Resultado verificado en el informe

a = 1

El analizador revisa ambas ramas sobre copias y propaga únicamente la elegida.

Política conservadora de las ramas

Ambas ramas deben ser válidas

```
producto a stock 0;
si a > 0 {
  salida a 1;
}
```

E006

El compilador rechaza el retiro aunque la condición sea falsa.

Recuperación	Conserva la primera declaración y el último stock válido. Una expresión errónea produce un valor desconocido.

Código intermedio y máquina virtual

Instrucción StockLang

```
entrada lapiz 2 * 3;
```

El compilador calcula 6 y suma esa cantidad al stock de lapiz.

La VM comprueba cantidades positivas, stock no negativo y límites enteros.

```
const t1, 2
const t2, 3
mul t3, t1, t2
load t4, lapiz
positive t3
add t5, t4, t3
store lapiz, t5
```

Notación legible. El IR real usa objetos JSON.

Plan de validación

01 **Aceptación y rechazo**

Casos válidos e inválidos por fase.

02 **Límites y regresión**

Fronteras enteras y casos del Hito 1.

03 **Ramas del condicional**

Casos verdaderos, falsos y anidados.

04 **Contraste independiente**

Cálculo manual y 80 programas con semilla fija.

Criterios: diagnóstico y posición correctos, sin IR inválido y stock consistente.

Resultados documentados

55

métodos unittest

0 fallos y 0 errores

13/13

casos de consola

Retorno y diagnóstico esperados

80

programas de movimientos

Oráculo con contador independiente

Demo **lapiz = 15** **cuaderno = 5**

30 muestras aceptadas y 24 derivaciones verificadas. Los 80 programas integran un método.

Limitaciones y seguimiento

Límite documentado	Efecto actual	Seguimiento propuesto
Validación conservadora	Rechaza retiros dentro de ramas que no se ejecutan.	Revisar el análisis del estado por rama.
Literal de 4301 dígitos	Python produce ValueError.	Emitir un diagnóstico controlado.

Alcance actual del lenguaje

El modelo usa enteros de 32 bits. No admite división ni signo unario.
Estas restricciones forman parte de la especificación del avance.

Conclusiones del Hito 2

El avance integra el análisis y la ejecución del inventario

01 **Lenguaje ampliado**

si / sino conserva el estado de la rama elegida.

02 **Reglas comprobadas**

Siete clases de error bloquean programas inválidos.